

Phase 3 Structured Note: Breadth-First Search (BFS)

1. Current Learning Position

You have already completed the foundation needed for Phase 3.

In **Phase 1**, you learned the basic language of graphs. You should already understand words like:

- graph
- vertex
- edge
- directed graph
- undirected graph
- degree
- indegree
- outdegree
- path
- cycle
- connected graph
- connected component
- DAG
- topological ordering

In **Phase 2**, you learned how to store graphs inside a C++ program. You should already understand:

- adjacency matrix
- adjacency list
- directed graph representation
- undirected graph representation
- weighted graph representation
- inverse adjacency list
- compact list
- matrix space complexity: $O(N^2)$
- list space complexity: $O(V + E)$

Now you are ready for **Phase 3: Breadth-First Search**, usually written as **BFS**.

Phase 3 answers this question:

Once a graph is stored in memory, how do we visit its vertices in an organized way?

BFS is your first graph traversal algorithm.

2. What Is Covered in Phase 3?

In this phase, you will learn:

1. What graph traversal means
2. What BFS means
3. Why BFS visits level by level
4. BFS intuition using a real-world example
5. BFS compared to tree level-order traversal
6. BFS spanning tree
7. Cross edges
8. Multiple valid BFS orders
9. Visiting vs exploring
10. The golden rule of BFS
11. Why BFS uses a queue
12. What FIFO means
13. Why BFS needs a visited array
14. BFS using an adjacency matrix
15. BFS using an adjacency list
16. BFS dry run from vertex
17. BFS dry run from vertex
18. Time complexity of BFS
19. Space complexity of BFS
20. Common beginner mistakes
21. Complete C++ BFS programs
22. Practice questions
23. Final checklist

3. Big Idea of Graph Traversal

A graph traversal means visiting the vertices of a graph in a planned order.

For example, suppose we have this graph:

```
1 ---- 2
|      |
3 ---- 4
```

The vertices are:

```
1, 2, 3, 4
```

A traversal might visit them in this order:

1 2 3 4

Or in another valid order:

1 3 2 4

The goal is not random movement. The goal is to follow a rule.

BFS follows this rule:

Visit nearby vertices first, then farther vertices later.

4. What Is BFS?

BFS stands for:

Breadth-First Search

The word **breadth** means width.

So BFS explores the graph by going outward in layers.

Simple meaning:

BFS starts from one vertex, visits all nearby vertices first, then moves farther away level by level.

5. BFS Intuition

Imagine you are standing at one city.

First, you ask:

Which cities can I reach using one road?

Then you ask:

Which cities can I reach using two roads?

Then:

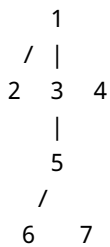
Which cities can I reach using three roads?

That is how BFS thinks.

It does not go very deep immediately. It expands outward step by step.

6. BFS as Level-by-Level Search

Suppose we have this graph:



If BFS starts at vertex **1**, it visits vertices by levels.

Level 0

The starting vertex:

1

Level 1

Vertices directly connected to **1**:

2, 3, 4

Level 2

Vertices two edges away from **1**:

5

Level 3

Vertices three edges away from 1:

6, 7

So one BFS order is:

1 2 3 4 5 6 7

Important idea:

BFS finishes one level before moving to the next level.

7. BFS Compared to Tree Level-Order Traversal

In trees, there is a traversal called **level-order traversal**.

Example tree:

```
  1
 / |
2 3 4
```

Level-order traversal visits:

1 2 3 4

BFS is similar, but BFS works on graphs.

The difference is that graphs may have cycles.

Example graph with a cycle:

```
1 ---- 2
|      |
4 ---- 3
```

A tree does not usually have cycles, but a graph can.

Because graphs can have cycles, BFS needs a **visited array** to avoid visiting the same vertex again and again.

8. What BFS Needs

BFS needs three main things:

Item	Purpose
Graph representation	Stores the edges of the graph
Visited array	Remembers which vertices were already seen
Queue	Controls the level-by-level order

The graph representation may be:

- adjacency matrix
- adjacency list

The queue is what makes BFS work level by level.

9. The Queue in BFS

BFS uses a **queue**.

A queue follows the rule:

First In, First Out

This is also called:

FIFO

Simple example:

Queue: [2, 3, 4]

The first item inside is 2, so 2 will be removed first.

After removing 2:

```
Queue: [3, 4]
```

This is like a line of people waiting.

The first person who joins the line is served first.

10. Why BFS Uses a Queue

BFS visits vertices in the order they are discovered.

Suppose BFS starts at **1** and discovers:

```
2, 3, 4
```

Then BFS should explore **2** before **3**, and **3** before **4**, because that is the order in which they were discovered.

A queue naturally supports this.

When a vertex is discovered, BFS puts it at the back of the queue.

When BFS needs the next vertex to explore, it removes from the front of the queue.

That is why BFS uses a queue.

11. The Visited Array

A **visited array** stores whether a vertex has already been visited.

Example:

```
vector<int> visited(n + 1, 0);
```

Here:

```
0 means not visited  
1 means visited
```

If the graph has vertices **1** to **7**, the visited array may look like this at the beginning:

```
visited[1] = 0
visited[2] = 0
visited[3] = 0
visited[4] = 0
visited[5] = 0
visited[6] = 0
visited[7] = 0
```

After visiting vertex **1**:

```
visited[1] = 1
```

12. Why BFS Needs a Visited Array

Graphs can contain cycles.

Example:

```
1 ---- 2
|      |
4 ---- 3
```

If we start at **1**, we can go:

```
1 -> 2 -> 3 -> 4 -> 1
```

Then we are back at **1** again.

Without a visited array, the program may keep repeating:

```
1 -> 2 -> 3 -> 4 -> 1 -> 2 -> 3 -> 4 -> ...
```

This could become an infinite loop.

So BFS uses this rule:

When you discover a vertex for the first time, immediately mark it as visited.

13. Visiting vs Exploring

BFS uses two related but different ideas:

Word	Simple meaning
Visiting	Seeing a vertex for the first time
Exploring	Checking all neighbors of that vertex

Example:

1 connected to 2, 3, 4

When BFS starts:

Visit 1

Then BFS explores 1:

Check all neighbors of 1
Discover 2, 3, 4
Visit 2, 3, 4
Put 2, 3, 4 into the queue

So:

- 1 was visited when BFS first reached it.
- 1 was explored when BFS checked its neighbors.

14. Golden Rule of BFS

The golden rule of BFS is:

Completely explore the current selected vertex before moving to the next vertex in the queue.

If BFS removes vertex 1 from the queue, it must check all neighbors of 1 before removing the next vertex.

Example:

Queue: [1]

Remove 1.

If 1 has neighbors 2, 3, 4, BFS checks all of them.

Then the queue becomes:

Queue: [2, 3, 4]

Only after finishing vertex 1 does BFS move to vertex 2.

15. BFS Spanning Tree

When BFS runs on a connected graph, it can create a tree-like structure called a **BFS spanning tree**.

Simple meaning:

A BFS spanning tree shows which edges BFS used to discover new vertices.

Suppose the graph has many edges:

```
1 ---- 2
|  \   |
3 ---- 4
```

There may be more than one way to reach a vertex.

BFS chooses the first edge that discovers a new vertex.

Those discovery edges become part of the BFS tree.

Edges that connect already discovered vertices are not used as tree edges.

16. Tree Edges and Cross Edges

During BFS, edges can be understood in two simple categories.

Edge type	Meaning
Tree edge	Edge used to discover a new vertex
Cross edge	Edge that connects to a vertex already discovered

Example:

```
1 discovers 2
1 discovers 3
2 also connects to 3
```

If vertex **3** was already discovered from **1**, then the edge **2--3** does not discover a new vertex.

So **2--3** is a cross edge.

Simple meaning:

A cross edge is an extra graph edge that BFS does not need in the BFS tree.

In BFS, cross edges usually connect vertices on the same level or nearby levels.

17. Multiple BFS Orders Can Be Correct

BFS does not always have one exact answer.

The BFS order can change depending on:

1. The starting vertex
2. The order in which neighbors are checked
3. The graph representation
4. The order edges were inserted into an adjacency list

Example graph:

```

      1
     / |
    2 3 4
```

If we check neighbors in increasing order, BFS from **1** may be:

1 2 3 4

If we check neighbors in reverse order, BFS from 1 may be:

1 4 3 2

Both are valid if BFS still follows the level-by-level rule.

Important beginner rule:

Do not memorize only one BFS output. Understand the rule that produced the output.

18. BFS Algorithm in Simple Steps

Suppose we start BFS from vertex `start`.

The steps are:

1. Create a visited array and set all values to 0.
2. Create an empty queue.
3. Visit the starting vertex.
4. Mark the starting vertex as visited.
5. Put the starting vertex into the queue.
6. While the queue is not empty:
 - a. Remove the front vertex from the queue.
 - b. Explore that vertex.
 - c. Check all of its neighbors.
 - d. If a neighbor is not visited:
 - Visit it.
 - Mark it visited.
 - Put it into the queue.
7. Stop when the queue becomes empty.

Short version:

```
Visit start.  
Push start into queue.  
While queue is not empty:  
    remove front vertex
```

```
visit all unvisited neighbors
push those neighbors into queue
```

19. BFS Pseudocode

```
BFS(start):
    create visited array
    create queue

    visited[start] = true
    print start
    enqueue start

    while queue is not empty:
        u = dequeue

        for each neighbor v of u:
            if visited[v] == false:
                visited[v] = true
                print v
                enqueue v
```

This pseudocode works with both adjacency matrix and adjacency list.

The difference is how we find the neighbors.

20. BFS Using an Adjacency Matrix

In an adjacency matrix:

```
matrix[u][v] = 1 means edge u -> v exists
matrix[u][v] = 0 means edge u -> v does not exist
```

For an undirected graph:

```
matrix[u][v] = 1;
matrix[v][u] = 1;
```

To find all neighbors of `u`, BFS scans the whole row:

```

for (int v = 1; v <= n; v++) {
    if (matrix[u][v] == 1) {
        // v is a neighbor of u
    }
}

```

This is simple, but it may scan many zeros.

21. BFS C++ Program Using an Adjacency Matrix

```

#include <iostream>
#include <vector>
#include <queue>
using namespace std;

void BFS(const vector<vector<int>>& graph, int start, int n) {
    vector<int> visited(n + 1, 0);
    queue<int> q;

    cout << start << " ";
    visited[start] = 1;
    q.push(start);

    while (!q.empty()) {
        int u = q.front();
        q.pop();

        for (int v = 1; v <= n; v++) {
            if (graph[u][v] == 1 && visited[v] == 0) {
                cout << v << " ";
                visited[v] = 1;
                q.push(v);
            }
        }
    }
}

int main() {
    int n = 7;
    vector<vector<int>> graph(n + 1, vector<int>(n + 1, 0));

    auto addEdge = [&](int u, int v) {
        graph[u][v] = 1;
        graph[v][u] = 1;
    };
}

```

```
};

addEdge(1, 2);
addEdge(1, 3);
addEdge(1, 4);
addEdge(3, 5);
addEdge(5, 6);
addEdge(5, 7);

cout << "BFS from vertex 1: ";
BFS(graph, 1, n);

return 0;
}
```

Expected output:

```
BFS from vertex 1: 1 2 3 4 5 6 7
```

22. Matrix Code Explanation

This line creates the visited array

```
vector<int> visited(n + 1, 0);
```

All vertices start as unvisited.

The program uses `n + 1` because the vertices are numbered from `1` to `n`.

Index `0` is ignored.

This line creates the queue

```
queue<int> q;
```

The queue stores vertices waiting to be explored.

These lines visit the starting vertex

```
cout << start << " ";  
visited[start] = 1;  
q.push(start);
```

Meaning:

```
Print the starting vertex.  
Mark it as visited.  
Put it into the queue.
```

This loop continues until BFS is finished

```
while (!q.empty())
```

BFS continues while the queue still has vertices waiting for exploration.

These lines remove the front vertex

```
int u = q.front();  
q.pop();
```

`u` is the current vertex being explored.

This loop scans possible neighbors

```
for (int v = 1; v <= n; v++)
```

Because we are using a matrix, we check every possible vertex `v`.

This condition checks for an unvisited neighbor

```
if (graph[u][v] == 1 && visited[v] == 0)
```


This means:

```
There is an edge from u to v  
AND  
v has not been visited yet
```

These lines visit the neighbor

```
cout << v << " ";  
visited[v] = 1;  
q.push(v);
```

Meaning:

```
Print v.  
Mark v as visited.  
Put v into the queue for later exploration.
```

23. BFS Dry Run from Vertex 1

Use this graph:

```
    1  
  / |  
2  3 4  
  |  
  5  
 /  
6  7
```

Start BFS from vertex **1**.

At the beginning:

```
visited[1] = 1  
queue = [1]  
output = 1
```

Now BFS begins.

Step	Action	Queue after action	Output so far
Start	Visit 1, mark it visited, enqueue 1	[1]	1
1	Dequeue 1. Discover 2, 3, 4. Visit and enqueue them.	[2, 3, 4]	1 2 3 4
2	Dequeue 2. No new unvisited neighbor.	[3, 4]	1 2 3 4
3	Dequeue 3. Discover 5. Visit and enqueue 5.	[4, 5]	1 2 3 4 5
4	Dequeue 4. No new unvisited neighbor.	[5]	1 2 3 4 5
5	Dequeue 5. Discover 6 and 7. Visit and enqueue them.	[6, 7]	1 2 3 4 5 6 7
6	Dequeue 6. No new unvisited neighbor.	[7]	1 2 3 4 5 6 7
7	Dequeue 7. No new unvisited neighbor.	[]	1 2 3 4 5 6 7

Final BFS order:

1 2 3 4 5 6 7

The queue becomes empty, so BFS stops.

24. Understanding the Queue Movement

The most important part of BFS is the queue.

Watch how the queue changes:

[1]
[2, 3, 4]
[3, 4]
[4, 5]
[5]
[6, 7]
[7]
[]

This shows BFS moving level by level.

First it handles 1.

Then it handles 2, 3, and 4.

Then it handles 5.

Then it handles 6 and 7.

25. BFS Dry Run from Vertex 4

Now use this graph:

```
1 ---- 2
|       |
3 ---- 4 ---- 5
        |
        6
```

Edges:

```
1--2
1--3
2--4
3--4
4--5
4--6
```

Start BFS from vertex 4.

Neighbors of 4 are:

```
2, 3, 5, 6
```

If we scan neighbors from small number to large number, BFS visits:

```
4 2 3 5 6 1
```

Dry run:

Step	Action	Queue after action	Output so far
Start	Visit 4, mark it visited, enqueue 4	[4]	4
1	Dequeue 4. Discover 2, 3, 5, 6.	[2, 3, 5, 6]	4 2 3 5 6
2	Dequeue 2. Discover 1.	[3, 5, 6, 1]	4 2 3 5 6 1
3	Dequeue 3. Vertex 1 is already visited. No new vertex.	[5, 6, 1]	4 2 3 5 6 1
4	Dequeue 5. No new vertex.	[6, 1]	4 2 3 5 6 1
5	Dequeue 6. No new vertex.	[1]	4 2 3 5 6 1
6	Dequeue 1. Vertices 2 and 3 already visited.	[]	4 2 3 5 6 1

Final BFS order:

4 2 3 5 6 1

Important observation:

Vertex 1 appears later because it is not directly discovered until BFS explores vertex 2.

26. BFS Using an Adjacency List

An adjacency list stores only real neighbors.

Example graph:

```

1 ---- 2
|      |
3 ---- 4

```

Adjacency list:

```

1: 2, 3
2: 1, 4
3: 1, 4
4: 2, 3

```

BFS with an adjacency list does not scan every possible vertex.

It only checks the stored neighbors.

That is why adjacency lists are usually better for BFS on sparse graphs.

27. BFS C++ Program Using an Adjacency List

```
#include <iostream>
#include <vector>
#include <queue>
using namespace std;

void BFS(const vector<vector<int>>& adj, int start, int n) {
    vector<int> visited(n + 1, 0);
    queue<int> q;

    visited[start] = 1;
    q.push(start);
    cout << start << " ";

    while (!q.empty()) {
        int u = q.front();
        q.pop();

        for (int v : adj[u]) {
            if (visited[v] == 0) {
                visited[v] = 1;
                q.push(v);
                cout << v << " ";
            }
        }
    }
}

int main() {
    int n = 7;
    vector<vector<int>> adj(n + 1);

    auto addEdge = [&](int u, int v) {
        adj[u].push_back(v);
        adj[v].push_back(u);
    };

    addEdge(1, 2);
    addEdge(1, 3);
    addEdge(1, 4);
```

```

    addEdge(3, 5);
    addEdge(5, 6);
    addEdge(5, 7);

    cout << "BFS from vertex 1: ";
    BFS(adj, 1, n);

    return 0;
}

```

Expected output:

```
BFS from vertex 1: 1 2 3 4 5 6 7
```

28. Matrix BFS vs List BFS

Feature	Matrix BFS	List BFS
Graph storage	2D table	List of neighbors
Neighbor checking	Scans all vertices	Scans actual neighbors only
Good for	Dense graphs	Sparse graphs
Time complexity	$O(N^2)$	$O(V + E)$
Memory for graph	$O(N^2)$	$O(V + E)$
Beginner difficulty	Very direct	Usually more efficient

Simple rule:

Matrix BFS checks every possible neighbor. List BFS checks only real neighbors.

29. Time Complexity of BFS

The time complexity depends on the graph representation.

29.1 BFS with Adjacency Matrix

In a matrix, for every vertex, we may scan a full row.

If there are N vertices, one row has N cells.

In the worst case, BFS may scan rows.

So the time complexity is:

$O(N^2)$

Why?

Because:

$N \text{ vertices} \times N \text{ checks per vertex} = N^2 \text{ checks}$

29.2 BFS with Adjacency List

In an adjacency list, BFS checks only real edges.

Each vertex is visited once.

Each edge is considered through the neighbor lists.

So the time complexity is:

$O(V + E)$

Where:

V = number of vertices
 E = number of edges

30. Space Complexity of BFS

BFS uses extra memory for:

1. the visited array
2. the queue

The visited array can store values.

The queue can also hold up to vertices in the worst case.

So the extra space complexity is:

$O(N)$

If using V instead of N , we can write:

$O(V)$

Important note:

This space complexity is for BFS's extra memory. The graph itself also uses memory depending on whether it is stored as a matrix or list.

31. BFS and Shortest Path Idea

BFS is useful because it visits vertices by distance from the starting vertex.

In an unweighted graph, BFS can find the shortest number of edges from the start vertex to other vertices.

Example:

```
1 ---- 2 ---- 5
|
3 ---- 4
```

Starting from 1:

- vertices at distance 0: 1
- vertices at distance 1: 2, 3
- vertices at distance 2: 5, 4

This is why BFS is often used for shortest paths in unweighted graphs.

In Phase 3, the main focus is traversal, but this shortest-path idea is an important reason BFS is useful.

32. BFS for Connected Components

BFS starting from one vertex visits all vertices reachable from that start vertex.

If the graph is connected, one BFS can visit the whole graph.

Example connected graph:

```
1 ---- 2 ---- 3
```

BFS from 1 visits:

```
1 2 3
```

But if the graph is not connected:

```
1 ---- 2      3 ---- 4
```

BFS from 1 visits only:

```
1 2
```

It will not reach 3 or 4 because they are in a different connected component.

To visit every vertex in a disconnected graph, run BFS from every unvisited vertex.

33. BFS for a Disconnected Graph

Example:

```
1 ---- 2      3 ---- 4      5
```

Components:

```
{1, 2}  
{3, 4}  
{5}
```

If BFS starts from 1, it visits:

```
1 2
```

Then vertices 3, 4, and 5 are still unvisited.

So we can do:

```
for each vertex:
    if vertex is not visited:
        run BFS from that vertex
```

This can also count connected components.

34. BFS for All Components: C++ Idea

```
for (int i = 1; i <= n; i++) {
    if (visited[i] == 0) {
        BFS(i);
    }
}
```

Simple meaning:

```
Check every vertex.
If it has not been visited yet, it belongs to a new component.
Run BFS from it.
```

35. Complete BFS Program for Disconnected Graphs

```
#include <iostream>
#include <vector>
#include <queue>
using namespace std;

void BFS(const vector<vector<int>>& adj, vector<int>& visited, int start) {
    queue<int> q;

    visited[start] = 1;
    q.push(start);
    cout << start << " ";

    while (!q.empty()) {
        int u = q.front();
        q.pop();
```

```

        for (int v : adj[u]) {
            if (visited[v] == 0) {
                visited[v] = 1;
                q.push(v);
                cout << v << " ";
            }
        }
    }
}

int main() {
    int n = 5;
    vector<vector<int>> adj(n + 1);

    auto addEdge = [&](int u, int v) {
        adj[u].push_back(v);
        adj[v].push_back(u);
    };

    addEdge(1, 2);
    addEdge(3, 4);

    vector<int> visited(n + 1, 0);
    int components = 0;

    for (int i = 1; i <= n; i++) {
        if (visited[i] == 0) {
            components++;
            cout << "Component " << components << ": ";
            BFS(adj, visited, i);
            cout << endl;
        }
    }

    cout << "Total components: " << components << endl;

    return 0;
}

```

Expected output:

```

Component 1: 1 2
Component 2: 3 4
Component 3: 5
Total components: 3

```

36. Important BFS Rule: Mark Before Enqueue

One of the most important BFS rules is:

Mark a vertex as visited before putting it into the queue.

Correct:

```
visited[v] = 1;  
q.push(v);
```

Not recommended:

```
q.push(v);  
visited[v] = 1;
```

Even worse:

```
q.push(v);  
// mark visited much later
```

Why does this matter?

Because if you do not mark immediately, the same vertex may be added to the queue more than once.

37. Example of Marking Visited Too Late

Suppose 1 connects to 2 and 3, and both 2 and 3 connect to 4.

```
  1  
  /\n 2  3  
  \ /\n  4
```

If vertex 4 is not marked when first discovered, both 2 and 3 may put 4 into the queue.

Bad queue situation:

```
[4, 4]
```

Now vertex **4** appears twice.

To avoid this, mark **4** as visited immediately when it is discovered.

38. BFS Output Timing

Some BFS programs print a vertex when it is first discovered.

Example:

```
cout << v << " ";  
visited[v] = 1;  
q.push(v);
```

Other programs print a vertex when it is dequeued.

Both styles can work if the visited logic is correct.

For beginners, it is usually easier to print when visiting/discovering the vertex.

This matches the idea:

```
When I first see it, I print it.
```

39. BFS on Directed Graphs

BFS also works on directed graphs.

The difference is that BFS must follow arrow directions.

Example:

```
1 ---> 2 ---> 3
```

BFS from **1** visits:

1 2 3

But BFS from 3 visits only:

3

Why?

Because there is no arrow from 3 back to 2 or 1.

Important rule:

In directed graphs, BFS can only move in the direction of the edge.

40. Directed BFS Example

Directed graph:

```
1 ---> 2
1 ---> 3
3 ---> 4
```

Adjacency list:

```
1: 2, 3
2:
3: 4
4:
```

BFS from 1:

1 2 3 4

BFS from 2:

2

BFS from 3:

3 4

Direction changes reachability.

41. BFS on Undirected Graphs

In an undirected graph, edges work both ways.

Example:

1 ---- 2 ---- 3

BFS from 1 visits:

1 2 3

BFS from 3 visits:

3 2 1

Because the edges can be traveled in both directions.

Important rule:

For undirected graphs, remember to store both directions in the adjacency list or matrix.

42. Common Beginner Mistakes

Mistake 1: Forgetting the visited array

Wrong idea:

I can just keep following edges.

Problem:

Graphs may have cycles.

Correct idea:

```
Use visited[] to avoid repeating vertices.
```

Mistake 2: Marking visited too late

Wrong:

```
q.push(v);
```

Then mark later.

Correct:

```
visited[v] = 1;  
q.push(v);
```

Mark the vertex as visited as soon as it is discovered.

Mistake 3: Using a stack instead of a queue

BFS uses a queue.

```
queue<int> q;
```

A stack gives DFS-like behavior.

Memory trick:

```
BFS = Queue  
DFS = Stack or Recursion
```

Mistake 4: Expecting one unique BFS order

There can be multiple valid BFS orders.

The order depends on the starting vertex and neighbor order.

Correct idea:

```
Check whether the traversal is level by level.
```

Mistake 5: Forgetting to initialize visited

Wrong:

```
vector<int> visited;
```

Correct:

```
vector<int> visited(n + 1, 0);
```

Every vertex should begin as unvisited.

Mistake 6: Forgetting reverse edges in an undirected graph

Wrong for undirected graph:

```
adj[u].push_back(v);
```

Correct:

```
adj[u].push_back(v);  
adj[v].push_back(u);
```

Mistake 7: Adding reverse edges in a directed graph

Wrong for directed edge `u -> v`:

```
adj[u].push_back(v);  
adj[v].push_back(u);
```

Correct:

```
adj[u].push_back(v);
```

Mistake 8: Confusing visiting and exploring

Visiting means:

I found this vertex for the first time.

Exploring means:

I am checking all neighbors of this vertex.

Mistake 9: Thinking BFS goes deep first

BFS does not go as deep as possible immediately.

That is DFS.

BFS goes level by level.

43. Phase 3 Summary Table

Concept	Simple meaning
BFS	Breadth-First Search
Graph traversal	Visiting graph vertices in an organized order
Breadth first	Visit nearby vertices before far vertices
Level-by-level	Finish distance 1 before distance 2, and so on
Queue	FIFO structure used by BFS
FIFO	First In, First Out
Visited array	Prevents repeated visits
Visit	See a vertex for the first time
Explore	Check all neighbors of a vertex

Concept	Simple meaning
BFS spanning tree	Tree of discovery edges created by BFS
Tree edge	Edge used to discover a new vertex
Cross edge	Edge to a vertex already discovered
Multiple valid BFS orders	BFS output may differ depending on neighbor order
Matrix BFS time	$O(N^2)$
List BFS time	$O(V + E)$
BFS extra space	$O(V)$ or $O(N)$

44. BFS Memory Trick

Remember this:

BFS uses a Queue.
Queue means First In, First Out.
First discovered, first explored.

Another memory trick:

BFS spreads like water ripples.

It starts at one point and expands outward layer by layer.

45. Mini Practice Questions

Question 1

What does BFS stand for?

Answer:

Breadth-First Search

Question 2

What data structure does BFS use?

Answer:

Queue

Question 3

What does FIFO mean?

Answer:

First In, First Out

Question 4

Why does BFS need a visited array?

Answer:

To avoid visiting the same vertex again and to prevent cycles from causing repeated work or infinite loops.

Question 5

What is the BFS order from vertex 1 if the graph is:

```
  1
 / |
2 3 4
```

and neighbors are checked from small to large?

Answer:

1 2 3 4

Question 6

In BFS, should you mark a vertex visited before or after putting it into the queue?

Answer:

Before putting it into the queue.

Question 7

What is the time complexity of BFS using an adjacency matrix?

Answer:

$O(N^2)$

Question 8

What is the time complexity of BFS using an adjacency list?

Answer:

$O(V + E)$

Question 9

Can BFS have more than one valid output order?

Answer:

Yes. The order can change depending on the starting vertex and neighbor order.

Question 10

What is the difference between visiting and exploring?

Answer:

Visiting means seeing a vertex for the first time.
Exploring means checking all neighbors of that vertex.

46. Practice Dry Run

Try BFS on this graph:

```
1 ---- 2 ---- 5
|      |
3 ---- 4
```

Start from vertex **1**.

Assume neighbors are checked from small to large.

Step 1

Visit **1**.

```
queue = [1]
output = 1
```

Step 2

Explore **1**.

Neighbors of **1**:

```
2, 3
```

Visit them.

```
queue = [2, 3]
output = 1 2 3
```

Step 3

Explore 2.

Neighbors of 2:

1, 4, 5

1 is already visited.

Visit 4 and 5.

```
queue = [3, 4, 5]
output = 1 2 3 4 5
```

Step 4

Explore 3.

Neighbors of 3:

1, 4

Both are already visited.

```
queue = [4, 5]
output = 1 2 3 4 5
```

Step 5

Explore 4.

No new vertex.

```
queue = [5]
```

Step 6

Explore 5.

No new vertex.

```
queue = []
```

Final BFS order:

```
1 2 3 4 5
```

47. Final Phase 3 Checklist

You understand Phase 3 if you can explain these without looking:

- What BFS stands for
 - What graph traversal means
 - Why BFS is called breadth-first
 - How BFS visits level by level
 - Why BFS is similar to tree level-order traversal
 - Why graphs need a visited array
 - What a queue is
 - What FIFO means
 - Why BFS uses a queue
 - What visiting means
 - What exploring means
 - The golden rule of BFS
 - What a BFS spanning tree is
 - What tree edges are
 - What cross edges are
 - Why multiple BFS outputs can be valid
 - How BFS works with an adjacency matrix
 - How BFS works with an adjacency list
 - Why matrix BFS is $O(N^2)$
 - Why list BFS is $O(V + E)$
 - Why BFS extra space is $O(V)$
 - How to dry run BFS using a queue table
 - Why visited should be marked before enqueueing
 - How BFS behaves differently on directed and undirected graphs
 - How BFS can help find connected components
-

48. Final Phase 3 Takeaway

BFS is a graph traversal algorithm that visits vertices level by level.

It starts from one vertex, visits all directly connected vertices, then moves farther away one level at a time.

The heart of BFS is the queue.

The queue makes sure that the first discovered vertex is explored first.

The visited array prevents repeated visits and protects the algorithm from cycles.

The most important BFS pattern is:

```
Mark start visited.  
Put start into queue.  
While queue is not empty:  
    remove front vertex  
    check its neighbors  
    for each unvisited neighbor:  
        mark visited  
        put into queue
```

Simple memory line:

```
BFS = level-by-level graph traversal using a queue and visited array.
```

After this, you are ready for **Phase 4: Depth-First Search (DFS)**, where the traversal style changes from level-by-level exploration to deep branch-by-branch exploration.